Лабораторная работа 1

Генетический алгоритм

Задание

1. Откройте lr01_ga.m. В начале укажите фамилию в комментарии и номер variant от 1 до 4.

2. Заполните только три функции: G2 — турнирный отбор, G3 — скрещивание, G4 — мутация. Правило G2 указано над функцией; правила G3 и G4 для своего варианта — на следующей странице. Замените строки error своим кодом. Начальная популяция, новое поколение с элитой и остальные функции уже готовы.

3. Выполните весь файл. Сравните pm = 0.02, 0.10 и 0.30 на функциях Розенброка и Растригина. Программа сама проведёт по пять запусков для каждой настройки и функции: пять переменных, по 4000 вычислений функции на запуск.

Графики

Для каждой функции появляется один рисунок. Слева — медиана лучшего найденного значения по пяти запускам; справа — пять итогов каждой настройки и их медиана. По вертикали log10(1 + f): меньше — лучше. Сравните скорость улучшения и разброс результатов. Файлы результатов автоматически не создаются.

Вопросы на защите

1. Как G2 выбирает победителя? Что происходит при равенстве значений функции?

2. Как вычисляются два потомка в вашем варианте? Покажите расчёт одной координаты.

3. Чем pm отличается от величины изменения координаты? Что получится при pm = 0?

4. Зачем в готовом коде сохраняются две элитные особи?

5. Какой pm оказался удачнее для каждой функции? Насколько согласованы итоги пяти запусков? Почему выводы для двух функций могут различаться?

Правила вариантов

Каждый выполняет только свой вариант. Переменная variant задаёт номер опыта; сами операторы G3 и G4 нужно реализовать в пропусках.

Вариант 1: арифметическое скрещивание + гауссовская мутация.

Вариант 2: равномерное скрещивание + гауссовская мутация.

Вариант 3: арифметическое скрещивание + случайная замена.

Вариант 4: равномерное скрещивание + случайная замена.

G3. Скрещивание: [a, b] = crossPair(p, q, u)

p и q — родители, a и b — потомки; все векторы — строки 1×d. u содержит готовые равномерные числа из [0, 1). Общий цикл сам выбирает, выполнять ли скрещивание, с вероятностью pc.

Варианты 1 и 3. Возьмите одно α = u(1) на всю пару. Для каждой координаты j:
a(j) = α·p(j) + (1−α)·q(j),
b(j) = (1−α)·p(j) + α·q(j).
В обеих формулах используйте исходных родителей.

Варианты 2 и 4. Если u(j) < 0.5, положите a(j) = p(j), b(j) = q(j); иначе a(j) = q(j), b(j) = p(j). Решение принимается отдельно для каждой координаты.

G4. Мутация

Функция y = mutatePoint(x, lb, ub, pm, sigmaRel, u, z, v) получает потомка x, границы lb/ub и готовые случайные числа. Все векторы — строки 1×d. u и v равномерны на [0, 1), z содержит стандартные нормальные числа.

Начните с копии x. Изменяйте только координаты, для которых u(j) < pm. Остальные координаты сохраните.

Варианты 1 и 2. Прибавьте к выбранной координате:
sigmaRel·(ub(j)−lb(j))·z(j).
Параметр sigmaRel = 0.05 задаёт масштаб изменения относительно ширины области. Аргумент v не используется.

Варианты 3 и 4. Замените выбранную координату:
y(j) = lb(j) + (ub(j)−lb(j))·v(j).
Аргументы sigmaRel и z не используются.

В конце G4 примените готовую clipPoint(y, lb, ub), чтобы точка оставалась в допустимой области. Внутри G3 и G4 не генерируйте новые случайные числа.